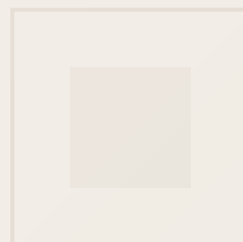
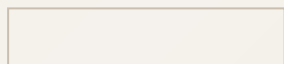


Claude Code Skill 完全指南

手把手教你在 Claude Code 中安装和使用 Skill 技能

作者：吕玉振

2026 年 7 月



目录

第1章 Skill 是什么

- 1.1 Skill 的基本概念
- 1.2 Skill 的核心特点
- 1.3 Skill 与 Plugin 的区别

第2章 怎样安装 Skill

- 2.1 Skill 文件应该放在哪里
- 2.2 安装别人的 Skill
- 2.3 创建自己的 Skill

第3章 怎样触发和使用 Skill

- 3.1 自动触发机制
- 3.2 手动触发
- 3.3 控制触发行为
- 3.4 查看和管理已安装的 Skill

第4章 去哪找更多 Skill

- 4.1 官方插件市场
- 4.2 第三方 Skill 分享网站
- 4.3 GitHub 开源社区
- 4.4 推荐的热门 Skill

附录

- A. Frontmatter 完整字段参考表
- B. 内置斜杠命令速查表
- C. 常见问题解答 (FAQ)

第1章 Skill是什么

Skill 是 Claude Code 最强大的扩展能力之一。它本质上是一套**可复用的指令模板**，让你能把 Claude Code 从一个通用 AI 助手，变成某个领域的**专业助手**——而且一次编写，到处使用。

1.1 Skill的基本概念

Skill（技能） 是 Anthropic 于 **2025年10月16日** 为 Claude Code 正式发布的功能，随后于 **12月18日**，Agent Skills 规范作为**开放标准**发布在 agentskills.io，目前已有 30+ 平台采纳支持。

Skill 的核心定义非常简单：一个**包含 SKILL.md 文件的目录，就是一个 Skill**。

这个 SKILL.md 文件由两部分组成：

组成部分	内容	作用
YAML Frontmatter	被 <code>---</code> 包裹的元数据	定义 Skill 的名称、描述、触发条件、可用工具等
Markdown 正文	指令和模板内容	告诉 Claude 在 Skill 激活时应该做什么、怎么做

用大白话说：Skill 就是你提前写好的"工作说明书"，告诉 Claude Code 在特定场景下如何更好地帮你完成任务。比如：

- 你可以写一个**代码审查 Skill**，让 Claude 每次审查代码时都按照统一的标准和格式
- 你可以写一个**Git 提交 Skill**，让 Claude 帮你生成规范化的 commit message
- 你可以写一个**技术文档 Skill**，让 Claude 帮你按照团队规范编写 API 文档

Skill 是如何工作的？

当你与 Claude Code 对话时，系统会检查当前可用的所有 Skill（根据目录层级确定），并读取每个 Skill 的 `description` 字段。如果你输入的内容与某个 Skill 的描述高度匹配，Claude Code 就会自动激活该 Skill，将 SKILL.md 中的指令注入到当前的对话上下文中，从而按照你预定义的规范来响应。

这个过程对用户是**完全透明**的——你只需要正常使用 Claude Code，合适的 Skill 会在恰当的时机自动生效。

核心原则： 写好一次，每次对话都能用；别人写好的，装上就能用。

1.2 Skill的核心特点

Skill 功能之所以强大，源于三个核心设计特点：

1. 可复用 (Write Once, Use Everywhere)

写好一个 Skill 之后，每次与 Claude Code 对话时都可以使用它，不需要每次都重复输入冗长的指令。

比如你有一个写单元测试的 Skill，一旦配置好，任何时候你说"帮我写测试"，Claude 都会自动按照你定义的模板和规范来生成，而不是每次都从零摸索你的偏好。

2. 可分享 (Share & Reuse)

Skill 本质上是文本文件 (SKILL.md)，可以很方便地：

- 通过 Git 仓库分享给团队成员
- 发布到 GitHub 上供社区使用
- 通过插件市场一键安装

这意味着你不需要从零开始——社区已经有大量现成可用的 Skill，装上就能提升效率。

3. 自动触发 (Auto-match & Manual Trigger)

Skill 有两种激活方式：

- **自动触发：** Claude Code 会根据你的输入内容，自动判断哪个 Skill 最合适并激活它。这个判断依据是 SKILL.md 中 `description` 字段的内容——**描述越准确，自动匹配率越高**。
- **手动触发：** 通过在输入框中输入**斜杠命令**（如 `/commit-msg`）来主动激活指定 Skill。

[提示] **小贴士：** `description` 字段是自动触发的灵魂，建议认真填写。前 **1536 个字符**会被系统读取，超出部分会被截断，所以要在这有限的篇幅内描述清楚 Skill 的适用场景。

1.3 Skill与Plugin的区别

初学者最容易混淆的三个概念是 **Skill**、**Plugin** 和 **MCP**。下面是清晰的对比：

Skill vs Plugin

维度	Skill（技能）	Plugin（插件）
本质	能力模块	容器/集合
结构	一个目录，核心是 <code>SKILL.md</code>	一个目录，核心是 <code>plugin.json</code>
内容	SKILL.md + 可选的 <code>scripts/</code> 、 <code>references/</code> 、 <code>assets/</code> 等	多个 Skill + <code>hooks</code> + <code>agents</code> + <code>MCP servers</code>
安装单位	单个 Skill	整个 Plugin（含多个组件）
使用方式	安装后自动或手动触发	安装后内部所有 Skill 同时生效

简单类比：

- **Skill** 像是一把螺丝刀（单一工具）
- **Plugin** 像是一个工具箱（包含螺丝刀、锤子、钳子等一整套工具）

安装一个 Plugin，等于一次性安装了其中包含的多个 Skill 和其他组件。

Skill vs MCP（Model Context Protocol）

维度	Skill	MCP
定位	流程层/知识层	外部连接层
作用	定义"怎么做"（工作流程、规范、模板）	定义"连接什么"（外部数据源、工具、API）
示例	代码审查规范、提交信息模板、文档格式	连接数据库、读取文件系统、调用外部 API

两者是互补协同的关系： Skill 告诉 Claude "按什么流程和规范来做事"，MCP 告诉 Claude "可以去哪里获取数据和使用什么外部工具"。在实际项目中，两者经常搭配使用。

[注意] **注意事项：**不要把 Skill 和 Plugin 混为一谈。Skill 是 Plugin 的组成部分，但 Skill 可以独立存在和使用，而 Plugin 是一个更大的容器概念。

第2章 怎样安装Skill

这一章介绍 Skill 的三种来源方式：安装别人写好的 Skill、创建自己的 Skill，以及 Skill 文件应该放在哪里才能被 Claude Code 正确识别。

2.1 Skill文件应该放在哪里

Claude Code 支持**四层目录结构**来存放 Skill，每一层对应不同的作用范围和优先级：

级别	存放路径	作用范围	适用场景
企业级	托管设置 (Managed Settings)	整个组织	公司统一规范，全员生效
用户级	<code>~/.claude/skills/<skill-name>/SKILL.md</code>	所有项目	个人常用 Skill，全局可用
项目级	<code>.claude/skills/<skill-name>/SKILL.md</code>	当前项目	项目特定的工作流和规范
插件级	<code><plugin>/skills/<skill-name>/SKILL.md</code>	随插件生效	安装插件时自动部署

优先级规则

当同名 Skill 存在于多个层级时，Claude Code 按照以下优先级加载（**优先级高的覆盖优先级低的**）：

企业级 (Enterprise) > 用户级 (Personal) > 项目级 (Project)

Monorepo 支持

对于大型单体仓库 (Monorepo)，Claude Code 还支持**嵌套的** `.claude/skills/` 目录。你可以在子项目目录下也创建 `.claude/skills/`，这些 Skill 只对所在子项目生效，实现了更精细的权

限控制。

目录结构示例

```
# 用户级 Skill 目录结构
~/.claude/
└─ skills/
    ├── commit-msg/
    │   └─ SKILL.md
    ├── code-review/
    │   └─ SKILL.md
    └─ doc-writing/
        └─ SKILL.md

# 项目级 Skill 目录结构
my-project/
├─ .claude/
│   └─ skills/
│       ├── deploy-guide/
│       │   └─ SKILL.md
│       └─ test-template/
│           └─ SKILL.md
├─ src/
└─ package.json
```

[提示] 小贴士：建议将**通用型 Skill**（如写 commit message、代码审查）放在 `~/.claude/skills/`（用户级），而将**项目专属 Skill**（如项目部署流程、团队编码规范）放在项目级的 `.claude/skills/` 目录下，并通过 Git 版本控制与团队共享。

2.2 安装别人的Skill

当你从 GitHub、同事或社区获取到一个现成的 Skill 时，有三种安装方式可选。

2.2.1 方式一：手动复制

最基础的方式，适用于所有 Skill。步骤如下：

1. **获取 Skill 文件夹** — 通过 `git clone` 或下载 ZIP 包
2. **复制到对应目录** — 根据需求选择用户级或项目级路径

```
# 第1步：从 GitHub 克隆 Skill 仓库
git clone https://github.com/author/skill-name.git

# 第2步：复制到用户级目录（所有项目可用）
cp -r skill-name ~/.claude/skills/

# 或者复制到项目级目录（仅当前项目可用）
cp -r skill-name .claude/skills/
```

安装完成后，Skill 立即生效，Claude Code 会根据匹配条件自动调用。

验证安装是否成功：

```
# 检查 Skill 目录结构是否正确
ls ~/.claude/skills/<skill-name>/
# 应该输出：SKILL.md

# 查看 SKILL.md 内容确认无误
cat ~/.claude/skills/<skill-name>/SKILL.md
```

[注意] **注意事项：** 确保复制的是整个 Skill 目录（包含 SKILL.md 的文件夹），而不是只复制 SKILL.md 单个文件。正确的目录结构应该是 `~/.claude/skills/my-skill/SKILL.md`，而不是 `~/.claude/skills/SKILL.md`。

[提示] **小贴士：** 如果你使用 Git 来管理自己的 dotfiles，可以将 `~/.claude/skills/` 目录加入版本控制，这样在新电脑上配置环境时，所有个人 Skill 都能一键同步。

2.2.2 方式二：用第三方工具一键安装

Vercel Labs 开发了一个便捷的 CLI 工具，可以通过命令行一键安装 Skill。

- **GitHub 仓库：** <https://github.com/vercel-labs/skills>
- **安装命令：**


```
# 基本语法
npx skills add <github-repo> --skill <skill-name>

# 示例：安装指定 Skill
npx skills add author/repo-name --skill my-skill

# 全局安装（放入 ~/.claude/skills/）
npx skills add <github-repo> --skill <skill-name> -g

# 项目级安装（放入 .claude/skills/）
npx skills add <github-repo> --skill <skill-name>
```

安装过程是**交互式**的，工具会引导你依次选择：

1. 选择 **Agent**（如 Claude Code）
2. 选择 **Scope**（全局或项目级）
3. 选择 **安装方式**（symlink 软链接 / copy 复制）

[提示] **小贴士**：你也可以使用 `bunx` 或 `pnpm dlx` 替代 `npx`，效果相同：

```
bunx skills add <github-repo> --skill <skill-name>
pnpm dlx skills add <github-repo> --skill <skill-name>
```

[注意] **注意事项**：该工具可能存在一个已知问题——有时不会自动创建 symlink（参考 GitHub Issue #851）。如果遇到 Skill 安装后无法触发的情况，可以尝试手动创建软链接：

```
ln -s /path/to/source/skill ~/.claude/skills/<skill-name>
```

2.2.3 方式三：安装插件

通过 Claude Code 的插件市场安装包含 Skill 的插件，适合安装官方或社区维护的 Skill 集合。

操作步骤：

```
# 第1步：添加插件市场
/plugin marketplace add anthropics/skills

# 第2步：安装指定插件（插件名@anthropic-agent-skills）
/plugin install 插件名@anthropic-agent-skills

# 示例：安装官方文档 Skill 插件
/plugin install document-skills@anthropic-agent-skills
```

安装完成后，插件内包含的所有 Skill 会自动生效，无需额外配置。

2.3 创建自己的Skill

如果现成的 Skill 无法满足你的需求，你可以轻松创建自己的 Skill。创建 Skill 有三种方式：手动编写、让 Claude Code 帮你生成、或使用 `/skill-creator`。

2.3.1 SKILL.md文件结构

SKILL.md 是 Skill 的核心文件。一个完整的 Skill 目录通常包含以下结构：

```
my-skill/
├── SKILL.md           # 核心文件（必需）
├── template.md        # 模板文件（可选）
├── examples/          # 示例目录（可选）
│   └── example1.md
├── scripts/           # 辅助脚本（可选）
│   └── helper.sh
├── references/         # 参考文档（可选）
│   └── guidelines.md
└── assets/            # 静态资源（可选）
    └── diagram.png
```

其中，只有 **SKILL.md** 是必需的，其他文件和目录都是可选的，根据 Skill 的复杂程度决定是否需要。

2.3.2 元信息（Frontmatter）详解

Frontmatter 是 SKILL.md 文件顶部被 `---` 包裹的 YAML 元数据区域，是 Skill 的"配置大脑"。完整支持 **15 个字段**：

字段	必需	类型	说明
<code>name</code>	否	string	显示名称，省略则用目录名。小写+数字+连字符，最多64字符
<code>description</code>	推荐	string	功能描述，Claude 据此判断何时加载 Skill。前1536字符有效
<code>when_to_use</code>	否	string	额外触发条件描述，追加到 <code>description</code> 后
<code>argument-hint</code>	否	string	自动补全时显示的参数提示
<code>arguments</code>	否	array	命名位置参数，用于 <code>\$name</code> 替换
<code>disable-model-invocation</code>	否	boolean	<code>true</code> = 禁止自动加载，仅手动触发（默认 <code>false</code> ）
<code>user-invocable</code>	否	boolean	<code>false</code> = 从 / 菜单隐藏，仅 Claude 能调用（默认 <code>true</code> ）
<code>allowed-tools</code>	否	array	Skill 激活时无需确认即可使用的工具列表
<code>model</code>	否	string	Skill 激活时使用的指定模型
<code>effort</code>	否	string	思考力度： <code>low</code> / <code>medium</code> / <code>high</code> / <code>xhigh</code> / <code>max</code>
<code>context</code>	否	string	设置为 <code>fork</code> 在独立 subagent 中运行
<code>agent</code>	否	string	<code>context: fork</code> 时使用的 subagent 类型
<code>hooks</code>	否	object	Skill 生命周期钩子（激活前/后执行的操作）
<code>paths</code>	否	array	Glob 模式，仅当匹配到指定文件时自动激活
<code>shell</code>	否	string	<code>!command</code> 使用的 shell: <code>bash</code> 或 <code>powershell</code>

最核心的字段：`description`

`description` 直接决定了自动触发的准确率。建议遵循以下原则：

- 明确说明 Skill 的功能
- 列举触发场景 ("当用户要求 XXX 时使用")
- 使用关键词描述适用领域
- 控制在 1536 字符以内（超出部分会被截断）

常用字段组合示例：

对于只希望在特定情况下自动激活的 Skill:

```
---
name: api-doc
description: 为 REST API 编写接口文档。当用户要求编写 API 文档、接口说明或更新接口定义时使用。
paths: ["**/openapi.yaml", "**/swagger.json", "**/routes/**"]
---
```

对于需要额外工具权限的 Skill:

```
---
name: deploy-check
description: 在部署前执行检查清单。当用户提到部署、上线、release 或 deploy 时使用。
allowed-tools: ["Bash", "Read", "Glob"]
effort: high
---
```

2.3.3 正文编写指南

Frontmatter 之后是 Markdown 正文部分，这里是 Skill 的“灵魂”——告诉 Claude 激活后应该做什么。

编写原则：

1. **指令清晰：**明确告诉 Claude 任务目标、步骤和输出格式
2. **格式规范：**使用 Markdown 的标题、列表、代码块来组织内容
3. **示例驱动：**提供输入输出示例，让 Claude 理解期望的结果样式
4. **约束明确：**指定字数限制、格式要求、风格偏好等约束条件
5. **适度长度：**正文不是越长越好，关键是信息密度高、指令无歧义。通常 50-200 行的正文已经足够描述大部分 workflow
6. **使用变量：**如果定义了 `arguments` 字段，可以在正文中用 `$变量名` 引用用户传入的参数

正文结构建议：

```
---  
# Frontmatter 区域  
---  
  
# 这里是正文  
  
## 任务目标  
简要说明这个 Skill 要做什么。  
  
## 操作步骤  
1. 第一步...  
2. 第二步...  
3. 第三步...  
  
## 输出格式  
指定期望的输出格式和结构。  
  
## 示例  
提供1-2个完整的输入/输出示例。
```

2.3.4 实用示例

下面是一个完整的 **Git 提交信息 Skill** 示例，展示了所有关键元素的实际运用：

```

---
name: commit-msg
description: 生成规范的Git提交信息。当用户要求写commit message、git commit、提交信息或提交代码时使用。
---

请根据当前的代码变更，生成一条规范的Git提交信息。

## 格式要求

1. 第一行用类型前缀，如 `feat`、`fix`、`docs`、`style`、`refactor`、`test`、`chore` 等
2. 简要描述改了什么，不超过 50 个字符
3. 如果改动较大，在下方补充详细说明，每行不超过 72 个字符
4. 使用中文描述

## 类型说明

| 前缀 | 用途 |
|-----|-----|
| feat | 新功能 |
| fix | Bug修复 |
| docs | 文档更新 |
| style | 代码格式调整（不影响功能） |
| refactor | 代码重构 |
| test | 测试相关 |
| chore | 构建/工具链/杂项 |

## 示例

feat: 添加用户登录功能

- 实现JWT token认证
- 添加登录页面表单验证
- 添加登录状态持久化

```

将这个文件保存为 `.claude/skills/commit-msg/SKILL.md`（或 `~/.claude/skills/commit-msg/SKILL.md`），就可以使用了。

[提示] 小贴士：在 `description` 中列举多个触发关键词（如"commit message"、"git commit"、"提交信息"），可以显著提高自动匹配的命中率。

2.3.5 让Claude帮你创建Skill

最简单的方式——直接在对话中描述你的需求，让 Claude Code 帮你生成 Skill。

对话示例：

用户：帮我创建一个写 Git 提交信息的 Skill

Claude：好的，我来为你创建一个 commit-msg Skill。

我会将它保存到 .claude/skills/commit-msg/SKILL.md

Claude 会自动：1. 分析你的需求 2. 编写合适的 Frontmatter（尤其是 `description`） 3. 编写清晰的正文指令 4. 将文件保存到正确的位置

提高生成质量的技巧：

- **描述要具体**：不要说"帮我写个文档 Skill"，而要说"帮我写一个 Skill，用于生成 REST API 接口文档，要求包含参数说明、返回值示例和错误码表格，使用 Markdown 格式"
- **给出示例**：告诉 Claude 你期望的输出样式，比如"参考我们团队的这种格式..."
- **指定触发条件**：明确告诉 Claude 在什么场景下应该激活这个 Skill

另一种方式：使用 `/skill-creator`

```
/skill-creator
```

[注意] 注意事项： `/skill-creator` 不是 Claude Code 内置命令，需要先通过插件安装。安装方式：

```
/plugin install skill-creator@anthropic-agent-skills
```

安装完成后，使用 `/skill-creator` 会进入一个交互式的 Skill 创建流程，引导你逐步完成配置。

[提示] 小贴士：即使是让 Claude 帮你创建，也建议在生成后检查一下 `description` 字段是否准确描述了你的使用场景。可以手动补充一些触发关键词，提高自动匹配的准确率。

小结

通过本章内容，你应该已经掌握了：

知识点	要点
Skill 的本质	包含 SKILL.md 的目录
目录层级	企业级 > 用户级(<code>~/.claude/skills/</code>) > 项目级(<code>.claude/skills/</code>) > 插件级
安装方式	手动复制、 <code>npx</code> 工具一键安装、安装插件
创建方式	手动编写、对话让 Claude 生成、 <code>/skill-creator</code>
Frontmatter	15 个可选字段， <code>description</code> 最重要
触发方式	自动匹配（依赖 <code>description</code> ）+ 手动斜杠命令

在下一章中，我们将学习如何在实际对话中**触发和使用 Skill**，包括自动触发机制的原理、斜杠命令的使用技巧、以及多个 Skill 同时生效时的优先级处理。

第3章 怎样触发和使用Skill

安装好 Skill 后，下一步就是学会如何调用它。Claude Code 提供了多种灵活的触发方式——既有自动化的智能匹配，也有精准的手动控制。本章将详细介绍 Skill 的触发机制、手动调用方法以及管理技巧。

3.1 自动触发机制

自动触发是 Claude Code Skill 系统最强大的特性之一。Claude 会持续监控对话内容，在适当的时候自动调用相关的 Skill。

触发原理

Claude 的自动触发遵循以下流程：

- 语义匹配：** Claude 读取所有已安装 Skill 的 `description` 字段，当用户输入的内容与某个 Skill 的描述匹配时，系统自动触发该 Skill。

2. **渐进式加载（Progressive Disclosure）**：为了节省 token，Claude 不会一次性加载所有 Skill 的完整内容，而是采用三级加载策略：

层级	加载内容	估算大小	触发条件
Level 1	Frontmatter（元数据）	~100 tokens/Skill	每次对话自动加载
Level 2	SKILL.md 正文	取决于 Skill 复杂度	description 匹配时加载
Level 3	链接的附加文件	取决于文件大小	实际需要时按需加载

[提示] **小贴士**：这种渐进式设计可以降低 **60%-70%** 的 token 消耗。如果你的项目安装了 20 个 Skill，按全量加载可能需要数千 tokens，而渐进式加载通常只需要几百 tokens。

如何提升自动触发准确率

自动触发的核心在于 `description` 字段的质量。以下是经过验证的优化建议：

- **基础版**：清晰的 `description` —— 触发率约 **20%**
- **优化版**：详细描述 + 触发场景说明 —— 触发率可提升到 **50%+**
- **进阶版**：添加 `examples` 字段提供示例 —— 触发率进一步提升到 **72%-90%**

```
# 触发率较低的描述
description: "帮助写代码"

# 触发率较高的描述
description: "在需要编写符合团队规范的 Git 提交信息时自动触发，支持 Angular 提交规范，能自动生成包含类型、作用域和描述的完整提交信息"

# 触发率最高的描述（含示例）
description: "在需要编写符合团队规范的 Git 提交信息时自动触发"
when_to_use: |
  - 用户说"帮我写提交信息"或"生成 commit message"
  - 执行 git commit 前需要格式化提交信息
  - 需要遵循 Conventional Commits 规范时
```

[注意] **注意事项**：`description` + `when_to_use` 的合计文本在 Skill 列表中会被截断至 **1536 个字符**。因此要把最关键的信息放在前面，避免重要内容被截断。

3.2 手动触发

当自动触发不够精准，或者你明确知道要使用哪个 Skill 时，可以通过斜杠命令手动触发。

基本语法

在对话输入框中输入 `/技能名` 即可手动触发对应的 Skill：

```
/commit-msg      # 触发写提交信息的 Skill
/debug           # 触发调试助手 Skill
/fix-issue 123    # 触发修复问题的 Skill，并传入参数 123
```

发现可用命令

按下 `/` 键会打开命令面板，列出所有可用的 Skill 和内置命令：

```
/                # 显示所有可用命令列表
/sk              # 继续打字可以筛选，比如输入 sk 会显示 skills 相关命令
```

常用内置斜杠命令

Claude Code 内置了以下常用命令（与 Skill 无关的内置命令）：

命令	说明
<code>/</code>	列出所有可用命令
<code>/skills</code>	列出所有已安装的 Skill
<code>/plugin</code>	打开插件管理界面
<code>/clear</code>	清除当前对话上下文
<code>/cost</code>	查看本次会话的 token 消耗和费用
<code>/help</code>	显示帮助信息

[提示] **小贴士：**当你不确定某个 Skill 的准确名称时，先输入 `/` 查看列表，然后继续打字进行筛选，这比反复尝试猜测名称要高效得多。

3.3 控制触发行为

通过 Frontmatter 中的两个关键字段——`disable-model-invocation` 和 `user-invocable`——你可以精细控制 Skill 的触发行为。

配置组合效果

Frontmatter 配置	用户可手动触发	Claude 可自动触发	description 是否加载
默认（无设置）	是	是	是
<code>disable-model-invocation: true</code>	是	否	否
<code>user-invocable: false</code>	否	是	是
两者都设置	否	否	否

使用场景建议

场景一：防止自动触发（推荐用于有副作用的操作）

```
---
name: deploy-to-production
description: 部署应用到生产环境
disable-model-invocation: true
---
```

适合场景：部署代码、提交到主分支、发送邮件、执行数据库迁移等具有实际副作用的操作。这样只有在用户**明确手动调用**时才会执行。

场景二：仅自动触发（推荐用于背景知识型 Skill）

```
---
name: legacy-system-docs
description: 公司内部遗留系统的架构说明和 API 规范
user-invocable: false
---
```

适合场景：遗留系统文档、API 规范参考、团队编码规范等。这类 Skill 不需要手动调用，Claude 会在相关对话中自动引用它们。

场景三：完全禁用

```
---
name: deprecated-skill
description: 已废弃的 Skill
disable-model-invocation: true
user-invocable: false
---
```

适合场景：临时禁用某个 Skill（比如发现 bug 或已被替代），无需删除文件即可快速停用。

[注意] **注意事项：** `disable-model-invocation: true` 不仅阻止自动触发，还会让 `description` 不被加载到上下文中。这意味着该 Skill 完全不会参与自动匹配过程，从而进一步节省 token。

3.4 查看和管理已安装的 Skill

查看所有已安装的 Skill

```
/skills # 专门列出所有已安装的 Skill，显示名称、来源和状态
```

输出示例：

```
已安装 Skills (5):
[项目] commit-msg      - Git 提交信息生成器
[项目] code-review     - 代码审查助手
[用户] caveman         - 极简 token 优化 Skill
[插件] document-skills - 文档处理工具包
[用户] react-best-practices - React 最佳实践
```

管理方法

操作	方法	说明
查看列表	<code>/skills</code> 命令	显示所有 Skill 及其来源
临时停用	修改 Frontmatter	添加 <code>disable-model-invocation: true</code> 和/或 <code>user-invocable: false</code>
删除手动安装的 Skill	删除文件夹	直接删除 <code>~/.claude/skills/</code> 或项目 <code>skills/</code> 目录下的对应文件夹
管理插件安装的 Skill	<code>/plugin</code> 命令	打开图形化插件管理界面
更新插件 Skill	<code>/plugin update <插件名></code>	更新到最新版本

优先级规则

当多个 Skill 可能同时触发时，Claude 按照以下优先级决定使用顺序：

企业级 (Enterprise) > 用户级 (User) > 项目级 (Project) > 插件级 (Plugin)

这意味着，如果你同时在用户目录和项目目录安装了同名 Skill，**项目级的 Skill 会优先生效。**

[提示] **小贴士：**定期运行 `/skills` 检查已安装的 Skill 列表，清理不再需要或过时的 Skill，可以有效降低对话的 token 消耗。

第4章 去哪找更多 Skill

Skill 生态正在快速发展，有多个渠道可以找到高质量的 Skill。本章将介绍官方市场、第三方平台、GitHub 社区以及值得推荐的热门 Skill。

4.1 官方插件市场

Anthropic 提供了官方的插件市场，这是获取高质量 Skill 的首选渠道。

访问方式

在 Claude Code 对话框中输入以下命令打开插件管理：

```
/plugin # 打开插件管理界面
/plugin marketplace add anthropics/skills # 添加官方市场
```

官方市场信息

属性	详情
市场名称	claude-plugins-official
GitHub 仓库	anthropics/claude-plugins-official
Skill 数量	36+（截至 2025 年 12 月）
特色插件	document-skills（文档处理）、example-skills（示例技能）

官方推荐插件

插件名	功能描述	适用场景
document-skills	支持 docx、pdf、pptx、xlsx 等文档的解析和处理	需要处理办公文档的项目
example-skills	官方示例 Skill 集合，可作为自定义 Skill 的参考	学习 Skill 编写规范

[注意] **注意事项：**使用 `/plugin` 命令需要 Claude Code 1.0.33+ 版本。如果遇到连接问题，请检查网络连接或尝试更新 Claude Code 到最新版本。

4.2 第三方 Skill 分享网站

除了官方市场，多个第三方平台也提供丰富的 Skill 资源。

网站	地址	状态	Skill 数量	特点
skills.sh	skills.sh	活跃	91,030+	开放生态系统，提供 <code>npx skills</code> CLI 工具，支持多平台
agentskill.sh	agentskill.sh	活跃	107,000+	最大的 Skill 目录，提供质量评分和安全审计
Lobehub Skills	lobehub.com/skills	活跃	285,095+	规模最大的 Skill 市场之一，支持 Claude、Codex、ChatGPT 多平台

使用第三方 Skill

以 skills.sh 为例，你可以通过 CLI 工具快速安装：

```
# 安装 skills.sh CLI
npx skills@latest

# 搜索 Skill
npx skills search commit-msg

# 安装指定的 Skill
npx skills install <skill-name>
```

[提示] 小贴士：第三方平台的 Skill 质量参差不齐。建议优先选择有评分、下载量高、最近有更新的 Skill。agentskill.sh 提供安全审计功能，可以帮助你评估 Skill 的可信度。

4.3 GitHub 开源社区

GitHub 是发现和分享 Skill 的重要平台。以下是值得关注的热门项目：

热门 Skill 仓库

项目	Stars	描述
affaan-m/everything-claude-code	165K	综合性能优化系统，包含大量实用 Skill
anthropics/skills	123K	官方 Skills 仓库，权威参考
forrestchang/andrej-karpathy-skills	80.5K	改进 Claude Code 行为的 Skill 集合
JuliusBrussee/caveman	44.8K	可减少 65% token 消耗的效率优化 Skill
hesreallyhim/awesome-claude-code	40.6K	精选 Claude Code 资源列表
daymade/claude-code-skills	19.4K	专业级 Skill 市场
alirezarezvani/claude-skills	-	156 个 Skills，支持 7 种工具

搜索技巧

在 GitHub 上搜索 Skill 时，尝试以下关键词组合：

```
claude code skill
claude code plugin
claude skill markdown
claude code skills
```

[提示] **小贴士：**Star 数高的项目通常质量更有保障。建议查看项目的 `README.md` 和最近提交记录，确保项目仍在维护中。

4.4 推荐的热门 Skill

以下是经过社区验证、值得优先尝试的热门 Skill：

通用效率类

Skill	来源	安装量	说明
find-skills	skills.sh	120 万+	在 Claude Code 内查找和管理 Skill，发现新 Skill 的利器
caveman	GitHub	-	极简主义 Skill，可减少 65% 的 token 消耗，适合大型项目

前端开发类

Skill	来源	安装量	说明
vercel-react-best-practices	插件市场	344.3K	React 最佳实践，包含组件设计和状态管理规范
frontend-design	插件市场	330.9K	前端设计规范和 UI 组件指导

文档处理类

Skill	来源	安装量	说明
document-skills	官方	-	支持 docx、pdf、pptx、xlsx 等文档格式解析

推荐组合

对于不同类型的开发者，以下是推荐的 Skill 组合：

前端开发者：

vercel-react-best-practices + frontend-design + caveman

全栈开发者：

commit-msg + document-skills + find-skills + caveman

DevOps/运维工程师：

deploy-to-production（自定义） + document-skills + caveman

[注意] **注意事项：**安装过多 Skill 会增加 token 消耗。建议保持 5-10 个核心 Skill，定期清理不常用的。caveman Skill 可以帮助你优化整体 token 使用效率。

附录

A. SKILL.md Frontmatter 完整字段参考表

以下表格列出了 SKILL.md Frontmatter 中所有可用的配置字段：

字段	类型	必填	说明	示例
<code>name</code>	string	是	Skill 的标识名称，用于手动触发	<code>name: commit-msg</code>
<code>description</code>	string	是	Skill 的功能描述，用于自动触发匹配	<code>description: 生成 Git 提交信息</code>
<code>tools</code>	array	否	Skill 需要的工具权限列表	<code>tools: [Read, Bash, Edit]</code>
<code>allowed-tools</code>	array	否	允许使用的工具（更细粒度控制）	<code>allowed-tools: [Read:0, Bash:1]</code>
<code>disable-model-invocation</code>	boolean	否	禁用 Claude 自动触发	<code>disable-model-invocation: true</code>
<code>user-invocable</code>	boolean	否	是否允许用户手动触发	<code>user-invocable: false</code>
<code>when_to_use</code>	string	否	补充触发场景说明（1536 字符限制）	见 3.1 节示例
<code>examples</code>	array	否	触发示例对话，提升触发准确率	<code>examples: ["帮我写提交信息"]</code>
<code>author</code>	string	否	Skill 作者信息	<code>author: your-name</code>
<code>version</code>	string	否	Skill 版本号	<code>version: 1.0.0</code>
<code>tags</code>	array	否	Skill 标签，便于分类和搜索	<code>tags: [git, workflow]</code>
<code>dependencies</code>	array	否	依赖的其他 Skill 或工具	<code>dependencies: [git-cli]</code>
<code>timeout</code>	number	否	Skill 执行超时时间（秒）	<code>timeout: 30</code>
<code>max_iterations</code>	number	否	最大执行迭代次数	<code>max_iterations: 10</code>

典型 Frontmatter 示例

```
---
name: code-review
description: 在需要审查代码质量时自动触发，提供详细的代码审查意见和改进建议
when_to_use: |
  - 用户说"帮我审查这段代码"
  - 提交 Pull Request 前需要代码检查
  - 需要评估代码质量和潜在问题
examples:
  - "帮我 review 这个函数"
  - "这段代码有什么问题吗"
  - "审查一下这个 PR"
tools: [Read, Glob]
disable-model-invocation: false
user-invocable: true
author: team-lead
version: 1.2.0
tags: [code-quality, review, workflow]
---
```

B. 内置斜杠命令速查表

命令管理

命令	说明
/	列出所有可用的斜杠命令和已安装 Skill
/help	显示帮助信息
/clear	清除当前对话上下文
/cost	查看本次会话的 token 消耗和费用

Skill 管理

命令	说明
<code>/skills</code>	列出所有已安装的 Skill（含来源和状态）
<code>/skill <name></code>	直接触发指定的 Skill

插件管理

命令	说明
<code>/plugin</code>	打开图形化插件管理界面
<code>/plugin list</code>	列出已安装的插件
<code>/plugin install <插件></code>	安装指定插件
<code>/plugin update <插件></code>	更新指定插件
<code>/plugin uninstall <插件></code>	卸载指定插件
<code>/plugin marketplace add <市场></code>	添加插件市场源
<code>/plugin marketplace list</code>	列出已添加的市场源

常用内置功能

命令	说明
<code>/debug</code>	触发调试助手，分析代码错误
<code>/fix</code>	触发修复助手，自动修复代码问题
<code>/test</code>	触发测试助手，生成或运行测试
<code>/simplify</code>	简化代码或文本
<code>/batch</code>	批量处理任务
<code>/loop</code>	循环执行指定操作
<code>/claude-api</code>	调用 Claude API 进行高级操作

[提示] **小贴士**：如果你安装了同名的 Skill，手动触发的优先级遵循 企业级 > 用户级 > 项目级 > 插件级 的规则。确保你触发的 Skill 是你期望的那个。

C. 常见问题解答 (FAQ)

Q1: Skill 和 MCP (Model Context Protocol) 有什么区别?

Skill 是**流程/知识层**，定义的是 Claude 如何思考和行动的指南——比如代码审查的步骤、提交信息的格式规范。Skill 是文本指令，告诉 Claude "在什么情况下该做什么"。

MCP 是**外部连接层**，定义的是 Claude 与外部工具和服务交互的协议——比如连接数据库、调用 API、操作文件系统。

两者是**互补关系**：Skill 决定 "做什么"，MCP 决定 "怎么连接到外部资源"。一个 Skill 可以通过 MCP 工具来完成实际的操作。

Q2: 为什么我的 Skill 没有自动触发?

最常见的原因是 `description` 字段不够清晰。Claude 依赖 `description` 来判断何时触发 Skill。

排查步骤：1. 检查 `description` 是否准确描述了触发场景 2. 添加 `when_to_use` 字段补充说明 3. 添加 `examples` 字段提供具体的触发对话示例 4. 确保 `disable-model-invocation` 没有被设置为 `true` 5. 使用 `/skills` 确认 Skill 已正确安装

Q3: Skill 的优先级怎么确定?

当多个同名或功能相似的 Skill 同时存在时，按以下优先级生效：

企业级 (Enterprise) > 用户级 (User) > 项目级 (Project) > 插件级 (Plugin)

项目级的 Skill 优先级高于全局 Skill，这意味着你可以为不同项目定制专属的 Skill。

Q4: 一个项目中可以同时使用多个 Skill 吗?

可以。Claude 能够同时自动调用多个 Skill。在一次对话中，Claude 可能会根据上下文自动激活 2-3 个相关的 Skill 协同工作。

例如，当你说 "帮我审查这段代码并生成提交信息" 时，Claude 可能同时触发 `code-review` 和 `commit-msg` 两个 Skill。

Q5: 创建 Skill 需要编程能力吗？

不需要。 创建 Skill 只需要会用 Markdown 编写文档。基本的 Skill 只需要一个 `SKILL.md` 文件，包含 Frontmatter 元数据和自然语言的指令内容。

当然，如果你想创建更复杂的 Skill（比如调用特定工具、处理复杂逻辑），了解一些基本的 YAML 配置和工具调用语法会有帮助。

Q6: Skill 安全吗？

Skill **本身是安全的**——它只是文本指令，不直接执行代码。Claude 会按照 Skill 中的指导来思考和行动。

但需要注意以下几点： - `allowed-tools` 字段要**谨慎配置**，只授予 Skill 真正需要的工具权限 - 安装来源不明的 Skill 时要**审查内容**，确保没有不当的操作指令 - 对于有副作用的操作（如部署、删除数据），建议设置 `disable-model-invocation: true`

Q7: 如何分享自己创建的 Skill？

分享 Skill 有多种方式：

- 1. **GitHub**：将 Skill 文件推送到 GitHub 仓库，其他人可以 clone 或下载使用
- 2. **第三方平台**：提交到 `skills.sh` 或 `agentskill.sh` 等平台
- 3. **团队共享**：将 Skill 放入项目的 `skills/` 目录，通过 Git 与团队成员共享

Q8: 为什么 `/plugin` 命令打不开市场？

可能的原因和解决方法：

原因	解决方法
Claude Code 版本过低	更新到 1.0.33+ 版本
网络连接问题	检查网络，尝试使用代理
市场源未添加	执行 <code>/plugin marketplace add anthropics/skills</code>
插件索引加载失败	重启 Claude Code 后重试

Q9: description + when_to_use 为什么被截断?

这是 Claude 的性能优化机制。所有 Skill 的 `description` + `when_to_use` 文本在加载到 Skill 列表时会被截断至 **1536 个字符**。超出部分不会参与触发匹配。

建议：把最重要的触发关键词和场景描述放在前面，确保在 1536 字符限制内。

Q10: 如何监控 Skill 的 token 消耗?

使用以下方法监控和优化：

```
/cost # 查看当前会话的 token 消耗
```

优化建议： - 精简 `description` 和 `when_to_use` 文本 - 使用 `caveman` Skill 减少整体 token 消耗 - 删除不再使用的 Skill - 对背景知识型 Skill 设置 `user-invocable: false`

[总结] **总结：**本章介绍了 Skill 的触发方式（自动/手动）、触发行为控制、管理方法，以及获取 Skill 的主要渠道（官方市场、第三方平台、GitHub 社区）。通过合理配置 Frontmatter 字段和选择高质量的 Skill，你可以显著提升 Claude Code 的工作效率。